PEOPLE'S DEMOCRATIC REPUBLIC OF ALGERIA
Ministry of Higher Education and Scientific Research
École Nationale Polytechnique
Automation and Control Department

INTERNSHIP REPORT

Deployment and ROS 2 Integration of a Reinforcement-Learning Walking Policy for the Unitree G1 EDU Humanoid

Simulation, command integration, and performance assessment

HOST INSTITUTION	Centre for the Development of Advanced Technologies (CDTA)
DIVISION / TEAM	Division Productique et Robotique (DPR)
PREPARED BY	Belmiloud Mohamed Amine
HOST SUPERVISOR	Mrs. Sara Bouraine, Team Leader
INTERNSHIP PERIOD	05 September 2026 – 30 September 2026

Academic year 2026–2027

Acknowledgements

I would like to express my sincere gratitude to my host supervisor, Mrs. Sara Bouraine, Team Leader at the Centre for the Development of Advanced Technologies (CDTA), for her guidance, support, and valuable insights throughout this internship. I also extend my gratitude to the entire team of the Division Productique et Robotique (DPR) for welcoming me into a stimulating research environment. Finally, I would like to thank my academic supervisors at École Nationale Polytechnique for their continuous encouragement and academic support.

Abstract

This report presents the work carried out during an internship at the Centre for the Development of Advanced Technologies (CDTA) on the locomotion of the Unitree G1 EDU bipedal humanoid. A **reinforcement-learning (RL) walking policy**, pre-trained and provided with the `unitree_rl_gym` repository, is run in the MuJoCo simulator by the original deployment script, which tracks a **velocity command (v_x, v_y, ω_z) fixed in a configuration file**. This script was extended into a **ROS 2 node** that receives the velocity command on the standard `/cmd_vel` topic and publishes **telemetry** (measured base velocity, joint states and policy inference time) for live monitoring. The report describes the control architecture, the observation and action interface of the policy, the execution model of the node, the monitoring tools and the tests performed. In a forward-walking test at 0.5 m/s in simulation, the **measured mean speed was 0.476 m/s** (RMSE 0.035 m/s); the simulation ran at about **5 % of real time** on the test machine.

Keywords: humanoid robotics, reinforcement learning, Unitree G1, MuJoCo, ROS 2, locomotion.

Contents

Acknowledgements **i**

Abstract **i**

List of Acronyms **vi**

1 Introduction **1**
1.1 Context . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives . . . 1
1.4 My Contribution . . . 1
1.5 Report Organization . . . 2

2 Presentation of the Host Institution (CDTA) **3**
2.1 Overview and Mission . . . 3
2.2 Organization . . . 3
2.3 Host Team . . . 4

3 Humanoid Robots and the Unitree G1 EDU **5**
3.1 Functional Hierarchy of an Autonomous Humanoid . . . 5
3.2 The Unitree G1 EDU Platform . . . 5

4 Modeling and Navigation Techniques **7**
4.1 Mathematical Paradigms for Bipedal Locomotion . . . 7
4.1.1 Full Rigid-Body Dynamics and Whole-Body Control . . . 7
4.1.2 Reduced-Order Models: LIPM and ZMP . . . 7
4.1.3 Reinforcement Learning . . . 7
4.2 Navigation Techniques . . . 8
4.3 Scope of the Project . . . 9

5 Prerequisites and Software Packages **10**
5.1 Development Environment . . . 10
5.2 Software Packages . . . 10
5.2.1 `unitree_rl_gym` . . . 10
5.2.2 `unitree_sdk2` and `unitree_ros2` . . . 10
5.3 Workspace Layout . . . 11

6 Standalone Deployment on the G1 in MuJoCo **12**
6.1 Overview of the Control Loops . . . 12
6.2 Observation Vector . . . 12

6.3 Policy Output and Joint Targets . . . 13
6.4 Low-Level PD Control . . . 13
6.5 Limits of the Standalone Version . . . 14

7 ROS 2 Integration for Control and Monitoring 15
7.1 Design Goal . . . 15
7.2 Overview of the Integrated Pipeline . . . 15
7.3 Data Flow and Execution Model . . . 16
7.4 Breakdown of the Execution Stages . . . 17
7.5 Monitoring . . . 19
7.6 Execution Procedure . . . 20

8 Tests and Results 22
8.1 Test Protocol . . . 22
8.2 Simulation Snapshots . . . 22
8.3 Velocity Tracking . . . 22
8.4 Timing Measurements . . . 23
8.5 Discussion . . . 23

9 Conclusion and Perspectives 25
9.1 Conclusion . . . 25
9.2 Perspectives . . . 25

Bibliography 27

List of Figures

1 Entrance of the CDTA. . . . 3
2 Organization of the CDTA. . . . 4
3 Four-layer functional hierarchy of autonomous humanoid locomotion. . . . 5
4 Mathematical modeling paradigms in humanoid locomotion. . . . 7
5 Navigation techniques and their link with the locomotion layers. . . . 8
6 Scope of the project and isolated part of the control pipeline. . . . 9
7 Software stack used in this project and components left out of scope. . . . 10
8 Control loops of the standalone deployment. . . . 12
9 Standalone architecture (before) versus ROS 2-integrated architecture (after). 15
10 High-level data flow and execution model. External asynchronous ROS 2 commands feed the policy evaluation (50 Hz), which drives the physics and motor control loop (500 Hz). . . . 16
11 Four-stage data flow architecture and telemetry integration pipeline. . . . 17
12 Bi-directional communication flow between ROS 2 command sources, the RL policy, MuJoCo physics, and monitoring tools. . . . 18
13 Global summary architecture of data flow and ROS 2 integration for the Unitree G1 in MuJoCo. . . . 19
14 Leg joint positions plotted live with `rqt_plot` during a forward walk. . . . 20
15 Node and topic graph of the system, from `rqt_graph`. . . . 20
16 MuJoCo frames for the three main commands. . . . 22
17 Commanded and measured velocity during the forward walk, plotted with `rqt_plot`. . . . 23

List of Tables

1 Main characteristics of the Unitree G1 EDU [3, 4]. 6
2 Structure of the 47-element observation vector $\mathbf{o}_t$. 13
3 Joint gains and default pose from `g1.yaml` (same values for each leg). 13
4 Other parameters of `g1.yaml` used by the deployment. 14
5 ROS 2 topics of `G1BridgeNode`. 19
6 Test scenarios. 22

List of Acronyms

CoM	Center of Mass	**PD**	Proportional–Derivative (control)
DDS	Data Distribution Service	**PPO**	Proximal Policy Optimization
DoF	Degrees of Freedom	**QP**	Quadratic Programming
IMU	Inertial Measurement Unit	**RL**	Reinforcement Learning
LIPM	Linear Inverted Pendulum Model	**ROS**	Robot Operating System
MDP	Markov Decision Process	**SLAM**	Simultaneous Localization and Mapping
MLP	Multi-Layer Perceptron	**URDF**	Unified Robot Description Format
MJCF	MuJoCo XML model format	**VLA**	Vision–Language–Action (model)
WBC	Whole-Body Control	**ZMP**	Zero Moment Point
DPR	Division Productique et Robotique		

CHAPTER 1

Introduction

1.1 Context

Humanoid robots are designed to operate in human environments, but walking on two legs remains a difficult control problem: the robot is under-actuated, contacts are intermittent, and **dynamic balance must be maintained at high frequency**. Classical model-based controllers are accurate but sensitive to modeling errors, which has led to a growing use of **RL-based locomotion policies trained in simulation**. The Unitree G1 EDU is a compact, research-oriented humanoid commonly used for this purpose.

1.2 Problem Statement

A locomotion policy is only useful if it can be commanded and monitored by the rest of a robotic system. In its original form, the deployment script runs the policy with a **velocity command fixed in a configuration file (`cmd_init`)**, which prevents integration with navigation or teleoperation nodes, and it **exposes no information about the running system**. The problem addressed in this internship is therefore: *how to run an existing RL walking policy for the G1 in simulation, command it through ROS 2, and observe its behavior, without disturbing the control loop?*

1.3 Objectives

- **Understand the architecture of a humanoid control stack** and position the work inside it.
- **Set up the simulation environment** (MuJoCo, PyTorch, ROS 2 Humble) and run the pre-trained policy of `unitree_rl_gym`.
- **Analyze the control pipeline**: observation, policy output and low-level PD control.
- **Integrate ROS 2** so that velocity commands are received on `/cmd_vel`.
- **Publish telemetry for monitoring**, and evaluate the behavior of the integrated system.

1.4 My Contribution

The walking policy (`motion.pt`), the MuJoCo robot model, the configuration file and the original deployment script `deploy_mujoco.py` come from the `unitree_rl_gym` repository [17]. **No policy training was performed in this work.** My contribution is the following:

1. installation of the environment (Ubuntu 22.04, ROS 2 Humble, MuJoCo, PyTorch) and execution of the original deployment;

2. analysis of the pipeline (observation vector, action scaling, PD control, loop rates);
3. extension of the original script into `deploy_mujoco_ros.py`: a ROS 2 node (`G1BridgeNode`) that subscribes to `/cmd_vel` and publishes three telemetry topics;
4. monitoring with `ros2 topic`, `rqt_plot` and `rqt_graph`, and tests with different velocity commands.

1.5 Report Organization

Chapter 2 presents the host institution. Chapter 3 introduces humanoid robots and the Unitree G1 EDU. Chapter 4 reviews modeling and navigation techniques and defines the scope of the project. Chapter 5 lists the prerequisites and software packages. Chapter 6 explains the standalone deployment, Chapter 7 the ROS 2 integration and monitoring, and Chapter 8 the tests. Chapter 9 concludes.

CHAPTER 2

Presentation of the Host Institution (CDTA)

2.1 Overview and Mission

The Centre for the Development of Advanced Technologies (CDTA) is a public scientific and technological research establishment (EPST) placed under the Ministry of Higher Education and Scientific Research (MESRS) and coordinated by the DGRSDT. Its origins date back to 1982, and it has operated under its current name since 1988 [1, 2].

The center conducts research, prototyping and technology transfer in robotics, artificial intelligence, telecommunications, optics and photonics, microelectronics, materials and embedded systems. Its cross-cutting priorities are health, energy, environment, water, digital technologies and industrial technologies [1].



Figure 1: Entrance of the CDTA.

2.2 Organization

The CDTA is led by a director and includes three technical departments (nine services), five research divisions with twenty-four teams, two research units in Sétif and four technology platforms [1].

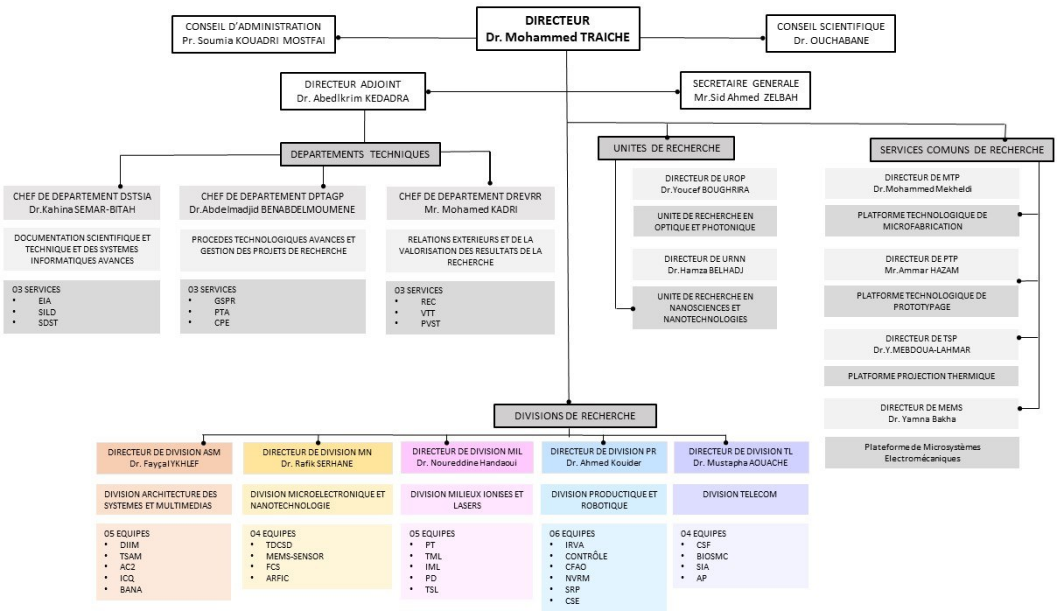


Figure 2: Organization of the CDTA.

2.3 Host Team

The internship was carried out in the Division Production et Robotique (DPR) of the CDTA, in the team led by Mrs. Sara Bouraine. The research of the team focuses on **mobile robotics**, and in particular on navigation: **localization, mapping and path planning** for robots that move in their environment.

This internship extends that context to a legged platform. In a navigation stack, the planner produces velocity commands, and a locomotion layer must turn them into stable motion. For a wheeled robot this layer is a standard motion controller; for a humanoid it is a learned walking policy. The work of this report provides the corresponding interface: the policy is commanded through the standard `/cmd_vel` topic, which is also the output of navigation stacks, so that the navigation methods developed in the team could in the future drive a humanoid.

CHAPTER 3

Humanoid Robots and the Unitree G1 EDU

3.1 Functional Hierarchy of an Autonomous Humanoid

The software of a fully autonomous bipedal humanoid is commonly organized in four layers, from abstract reasoning down to joint actuation (Figure 3).

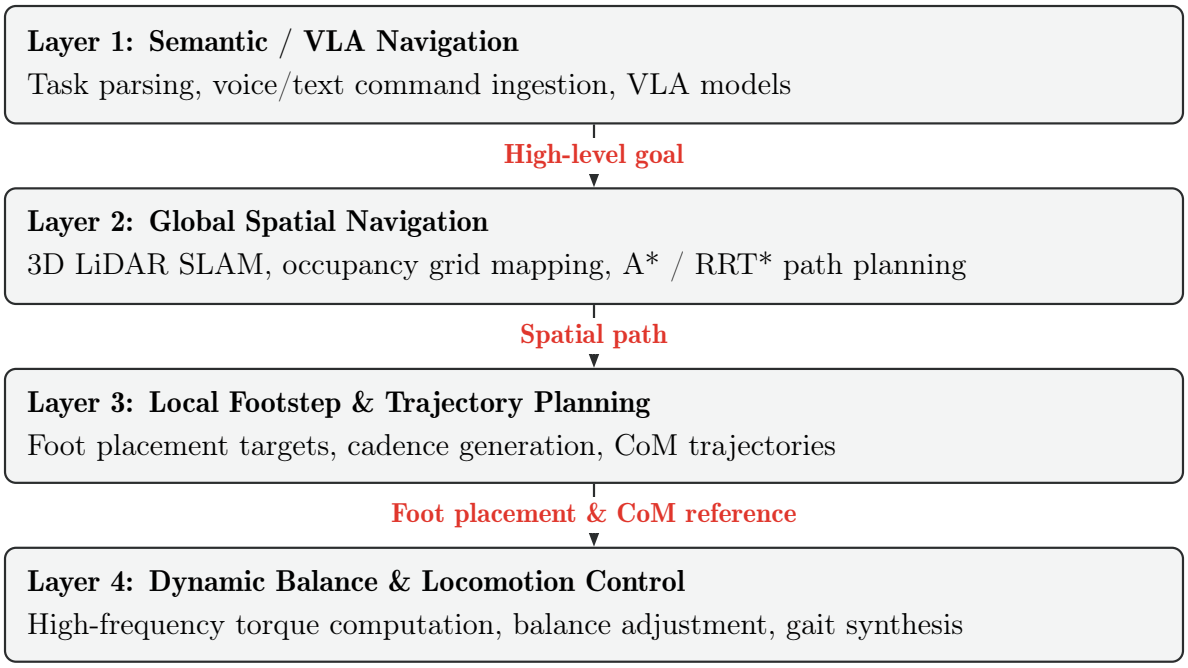


Figure 3: Four-layer functional hierarchy of autonomous humanoid locomotion.

- **Layer 1 (semantic):** converts natural-language commands or visual goals into navigation targets.
- **Layer 2 (global navigation):** builds a map (e.g. 3D LiDAR SLAM) and plans an obstacle-free path.
- **Layer 3 (footstep and trajectory planning):** chooses foot landing locations and CoM references from local terrain.
- **Layer 4 (balance and locomotion control):** generates high-frequency joint commands that keep the robot balanced, reject pushes and produce a stable gait.

3.2 The Unitree G1 EDU Platform

The Unitree G1 EDU is a compact bipedal humanoid intended for secondary development and mobility research. It supports up to 43 degrees of freedom depending on the configuration. This work uses only the 12 leg joints (6 per leg), the upper body being held in a fixed posture. Table 1 summarizes the main characteristics used in this report.

Table 1: Main characteristics of the Unitree G1 EDU [3, 4].

Parameter	**Value**	**Details**
Height / mass	1.32 m / ≈35 kg	Compact form factor, low CoM
Locomotion DoF	12 (2×6)	Hip pitch/roll/yaw, knee, ankle pitch/roll
Peak knee torque	120 N·m	Low-inertia permanent-magnet synchronous motors
Joint sensing	Dual encoders per joint	Absolute position and velocity measurement
Main sensor	6-axis pelvis IMU	Base orientation and angular rates
Onboard computer	NVIDIA Jetson Orin NX (100 TOPS)	8-core Cortex-A78AE @ 2.0 GHz, 16 GB RAM
Communication	Gigabit Ethernet / Wi-Fi 6	Unitree SDK2 over DDS

CHAPTER 4

Modeling and Navigation Techniques

4.1 Mathematical Paradigms for Bipedal Locomotion

Three families of approaches are used for bipedal balance. Figure 4 compares them.

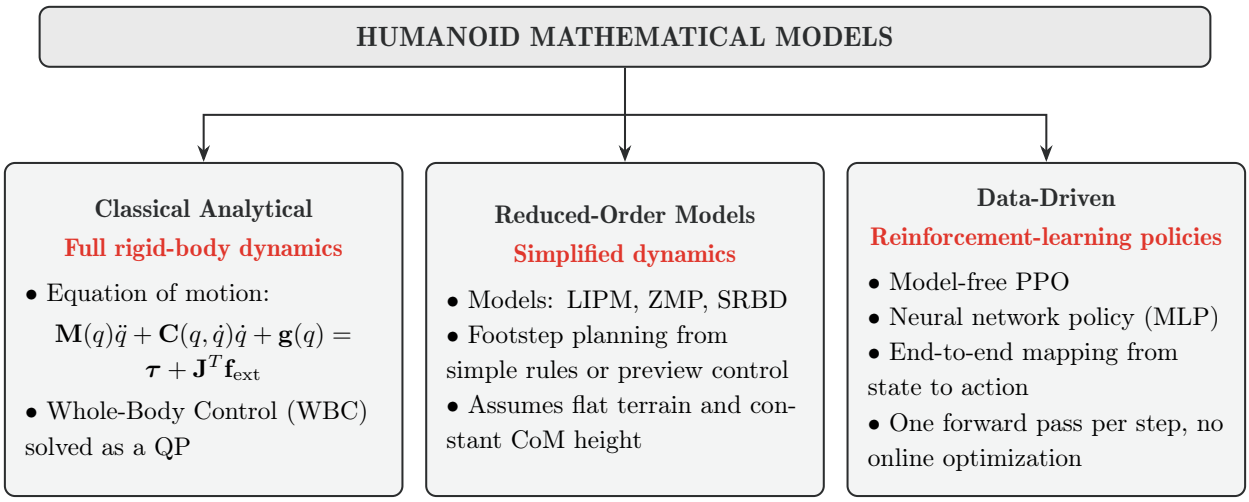


Figure 4: Mathematical modeling paradigms in humanoid locomotion.

4.1.1 Full Rigid-Body Dynamics and Whole-Body Control

The motion of the robot obeys

$$\mathbf{M}(q)\ddot{q} + \mathbf{C}(q,\dot{q})\dot{q} + \mathbf{g}(q) = \boldsymbol{\tau} + \mathbf{J}^T\mathbf{f}_{\text{ext}}, \tag{1}$$

where $\mathbf{M}$ is the inertia matrix, $\mathbf{C}$ the Coriolis term, $\mathbf{g}$ gravity and $\mathbf{J}^T\mathbf{f}_{\text{ext}}$ the contact forces. WBC solves a QP at each cycle to find the torques $\boldsymbol{\tau}$ satisfying task, torque and friction-cone constraints. It is accurate but **sensitive to unmodeled masses and friction, and computationally demanding**.

4.1.2 Reduced-Order Models: LIPM and ZMP

The humanoid is approximated by a point mass at constant height (LIPM). Balance is ensured by keeping the Zero Moment Point inside the support polygon of the stance foot [9]. The approach is simple but **assumes flat ground and rigid contacts**, and degrades on uneven terrain or during fast turns.

4.1.3 Reinforcement Learning

Locomotion is formulated as a Markov Decision Process: at each step the policy observes the state of the robot and chooses an action, and a reward function scores the result (for example, rewarding the tracking of a commanded velocity and penalizing falls and jerky motion). A neural network policy (an MLP) is optimized with a model-free algorithm such

as PPO [7] over millions of simulated steps in parallel environments [8]. This produces a **learned state-to-action mapping** that can be evaluated during deployment.

Training versus deployment. These are two distinct phases. *Training* is an offline optimization of the network weights by trial and error in simulation; it is computationally heavy (GPU, hours) and produces a weights file. *Deployment* (or inference) loads this file and only evaluates the network: at each control step, one forward pass maps the observation to joint targets, with **no learning and no online optimization**. This project covers deployment only; the policy file was trained beforehand by Unitree.

4.2 Navigation Techniques

Layers 1 and 2 of the hierarchy (Figure 3) answer the question *where should the robot go, and by which path?* Three families of techniques are used for this, summarized in Figure 5. Their common output is a motion request that the locomotion layers execute; in this project, that output is the **velocity command** (v_x, v_y, ω_z).

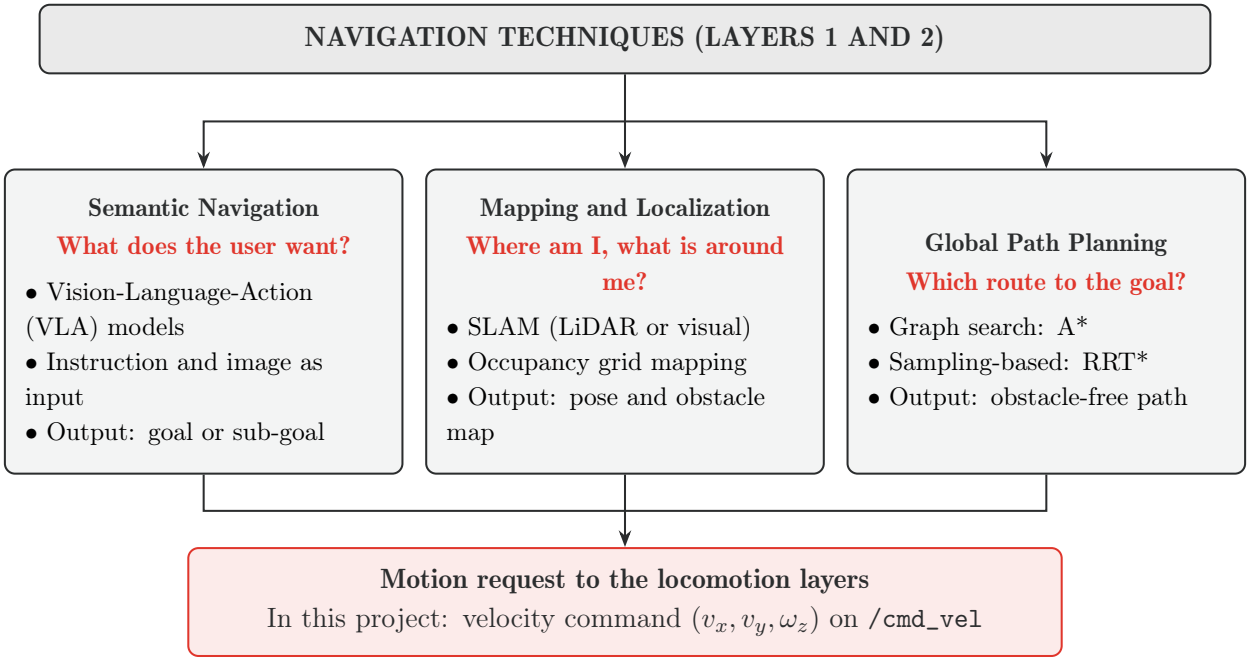


Figure 5: Navigation techniques and their link with the locomotion layers.

The techniques of Figure 5 are briefly described below. **SLAM** estimates the pose of the robot while building a map of an unknown environment [10, 11]. **Occupancy grids** store, for each cell of the map, the probability that it is occupied [12]. On such a map, **A*** [13] and **RRT*** [14] compute an obstacle-free path, by graph search and by random sampling respectively. At the highest level, **VLA models** take camera images and a natural-language instruction and output goals or actions [15, 16].

4.3 Scope of the Project

This work covers the **deployment** of a pre-trained policy that merges **Layers 3 and 4**, and uses a velocity command in place of Layers 1 and 2 (Figure 6).

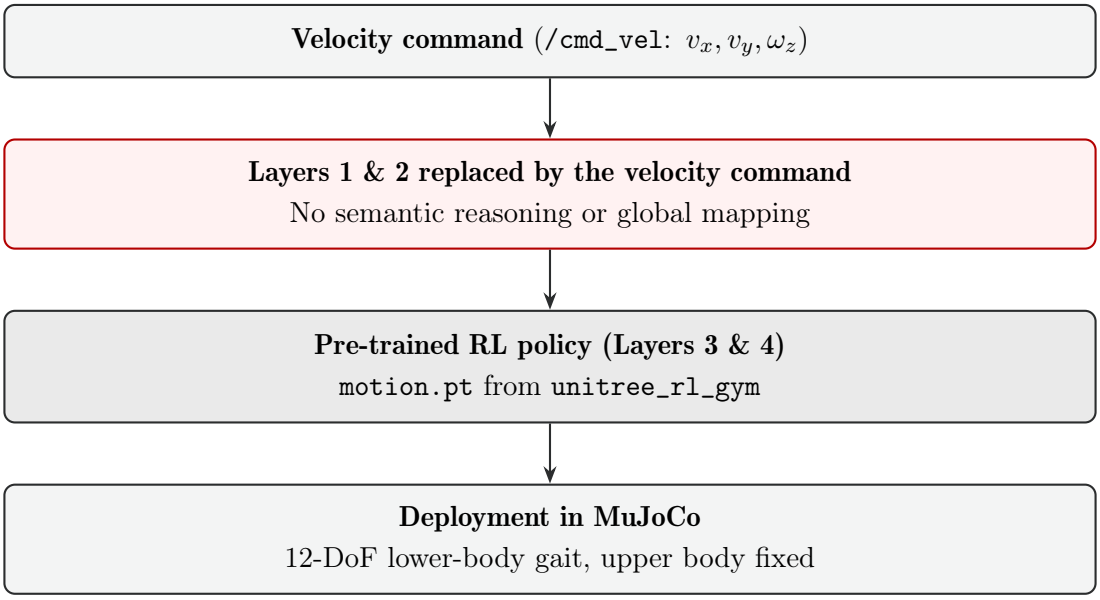


Figure 6: Scope of the project and isolated part of the control pipeline.

What this report does not cover: the training of the policy (reward design, PPO hyper-parameters, learning curves), which was done by Unitree, and the deployment on the physical robot.

CHAPTER 5

Prerequisites and Software Packages

5.1 Development Environment

The project uses only the deployment side of the Unitree software. Figure 7 separates what is used in this work from what belongs to policy training and to the real-robot interface.

USED IN THIS PROJECT

- Ubuntu 22.04 LTS (WSL2)
- Python 3, PyTorch (inference)
- MuJoCo 3.x
- ROS 2 Humble Hawksbill
- `unitree_rl_gym`: deployment script, configuration, pre-trained policy, robot model

NOT USED (OUT OF SCOPE)

- Isaac Gym and CUDA-enabled PyTorch: policy *training* (done by Unitree)
- `unitree_sdk2` and `unitree_ros2`: interface to the *real* robot over DDS (future work)

Figure 7: Software stack used in this project and components left out of scope.

- **ROS 2 Humble:** middleware for inter-process communication through topics such as `/cmd_vel` [5].
- **MuJoCo:** physics engine handling multi-body dynamics and contacts [6].
- **PyTorch:** loads the pre-trained network (TorchScript file) and evaluates it at each policy step.

5.2 Software Packages

5.2.1 `unitree_rl_gym`

Unitree's repository for training RL locomotion policies and deploying them in MuJoCo or on the robot [17]. Only the deployment part is used here:

- `deploy/deploy_mujoco/`: the original deployment script and the configuration files (here `g1.yaml`);
- `deploy/pre_train/g1/motion.pt`: the pre-trained walking policy;
- `resources/robots/g1_description/`: the MuJoCo (MJCF) model of the G1 and the simulation scene (`scene.xml`).

5.2.2 `unitree_sdk2` and `unitree_ros2`

Unitree's SDK and its ROS 2 wrapper, which define the G1 message types (`unitree_hg/msg/LowCmd`, `unitree_hg/msg/LowState`) and communicate with the real robot over DDS [18]. They are

not used in this simulation-only work. They would be required to send the same policy outputs to the physical G1 (see perspectives).

5.3 Workspace Layout

Only the files involved in this work are listed:

```
unitree_rl_gym/
+-- deploy/
|   +-- deploy_mujoco/
|   |   +-- deploy_mujoco.py        (original script)
|   |   +-- deploy_mujoco_ros.py    (ROS 2 version, this work)
|   |   +-- configs/g1.yaml
|   +-- pre_train/g1/motion.pt      (pre-trained policy)
+-- resources/robots/g1_description/
    +-- scene.xml                   (MuJoCo model and scene)
```

CHAPTER 6

Standalone Deployment on the G1 in MuJoCo

This chapter describes the deployment script *before* ROS 2 is added. The velocity command is fixed in the configuration file (`cmd_init`) and cannot change while the simulation runs.

6.1 Overview of the Control Loops

The script runs one main loop in which each iteration advances the physics by one step of $\Delta t = 0.002$ s, i.e. 500 Hz (`simulation_dt`). The policy is evaluated only once every `control_decimation` $= 10$ steps, i.e. every 20 ms (50 Hz). Between two policy evaluations, the joint targets are held constant and a PD controller keeps tracking them at every physics step (Figure 8).

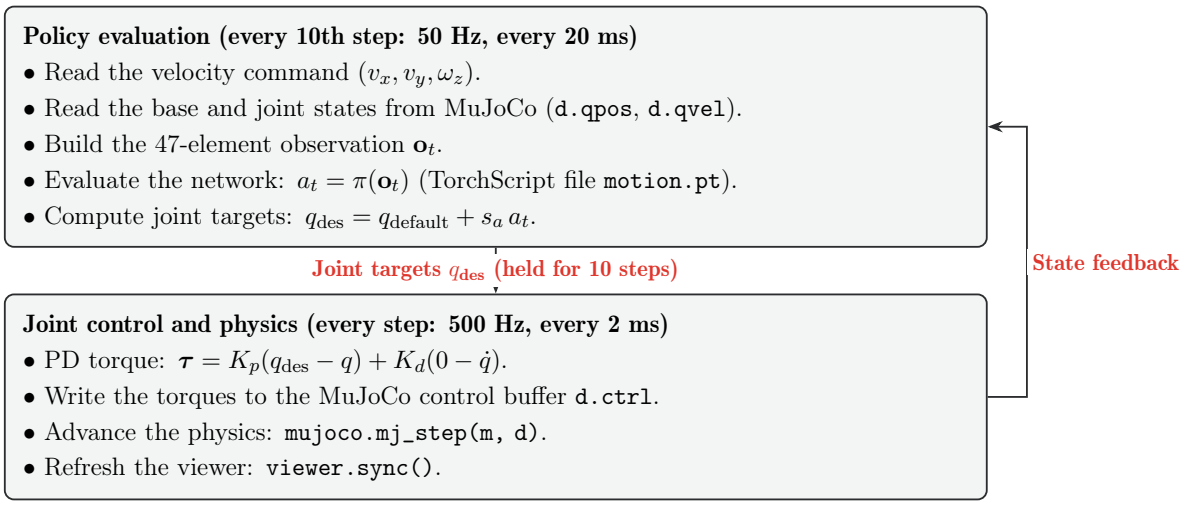


Figure 8: Control loops of the standalone deployment.

The policy is the "slow brain": it decides, 50 times per second, where each joint should go. The PD controller is the "fast reflex": it converts the remaining position error into a torque 500 times per second. The network never outputs torques directly.

6.2 Observation Vector

Every 20 ms, a **47-element vector** $\mathbf{o}_t \in \mathbb{R}^{47}$ is assembled (Table 2). It combines the base rotation and orientation, the commanded velocity, the leg joint state, the previous action and a sine/cosine gait clock. The network is a feed-forward MLP without memory: the previous action gives it context and promotes smooth motion, and the clock (period 0.8 s, fixed in the script) gives it a notion of time from which a periodic gait can emerge. Each group is multiplied by a scale factor (Table 4) so that the network receives values in the range used during its training.

Table 2: Structure of the 47-element observation vector $\mathbf{o}_t$.

Index	Component	Meaning and source	Dim.
$[0 \dots 2]$	Base angular velocity $\boldsymbol{\omega}_b$	Base rotation rate from the simulator state (IMU gyroscope on the real robot), scaled by 0.25	3
$[3 \dots 5]$	Projected gravity $\mathbf{g}_b$	Gravity direction in the base frame, computed from the base orientation quaternion	3
$[6 \dots 8]$	Velocity command $\mathbf{c}_t$	(v_x, v_y, ω_z) multiplied by $(2, 2, 0.25)$	3
$[9 \dots 20]$	Joint position offsets Δq	$q_{\text{measured}} - q_{\text{default}}$, 12 leg joints	12
$[21 \dots 32]$	Joint velocities $\dot{q}$	Joint velocities from the simulator, scaled by 0.05	12
$[33 \dots 44]$	Previous action a_{t-1}	Raw output of the previous policy step	12
$[45 \dots 46]$	Gait phase $[\sin\phi, \cos\phi]$	Clock with a period of 0.8 s, enforcing the stepping cadence	2

6.3 Policy Output and Joint Targets

The network outputs a vector $a_t \in \mathbb{R}^{12}$ (no clipping is applied in the script), converted to joint position targets by

$$a_t = \pi(\mathbf{o}_t), \qquad q_{\text{des}} = q_{\text{default}} + s_a\, a_t, \tag{2}$$

where $s_a = 0.25$ is the action scale and q_{default} is the nominal standing pose, i.e. the policy outputs offsets around the standing posture.

6.4 Low-Level PD Control

At each physics step, the torque applied to each joint is

$$\boldsymbol{\tau} = K_p(q_{\text{des}} - q) + K_d(0 - \dot{q}), \tag{3}$$

with the stiffness K_p and damping K_d of Table 3, identical for the left and right legs.

Table 3: Joint gains and default pose from `g1.yaml` (same values for each leg).

Joint	$\mathbf{K_p}$	$\mathbf{K_d}$	Default angle (rad)
Hip pitch	100	2	-0.1
Hip roll	100	2	0.0
Hip yaw	100	2	0.0
Knee	150	4	0.3
Ankle pitch	40	2	-0.2
Ankle roll	40	2	0.0

Table 4: Other parameters of `g1.yaml` used by the deployment.

Parameter	Value
`simulation_dt`	0.002 s (500 Hz physics)
`control_decimation`	10 (policy period 20 ms, 50 Hz)
`action_scale`	0.25
`ang_vel_scale` / `dof_pos_scale` / `dof_vel_scale`	0.25 / 1.0 / 0.05
`cmd_scale`	$(2.0, 2.0, 0.25)$
`num_obs` / `num_actions`	47 / 12
`cmd_init`	$(0, 0, 0)$

6.5 Limits of the Standalone Version

The velocity command is read once from the configuration file. It cannot be changed while the simulation runs and cannot be sent by another node, which prevents connection to navigation or teleoperation tools, and the script does not report anything about the running system. This motivates the ROS 2 integration of Chapter 7.

CHAPTER 7

ROS 2 Integration for Control and Monitoring

7.1 Design Goal

The objective of this integration is twofold:

1. **Control path:** receive velocity commands from any ROS 2 navigation or teleoperation node via the standard `/cmd_vel` topic (`geometry_msgs/msg/Twist`) without disturbing the physics loop. The linear speeds (v_x, v_y) and the yaw rate (ω_z) form the policy command.

2. **Monitoring path:** publish the measured base velocity, the joint states and the policy inference time back to ROS 2, to enable live visualization with standard tools such as `rqt_plot` and `rqt_graph`.

Figure 9 compares the original standalone architecture with the integrated ROS 2 bridge.

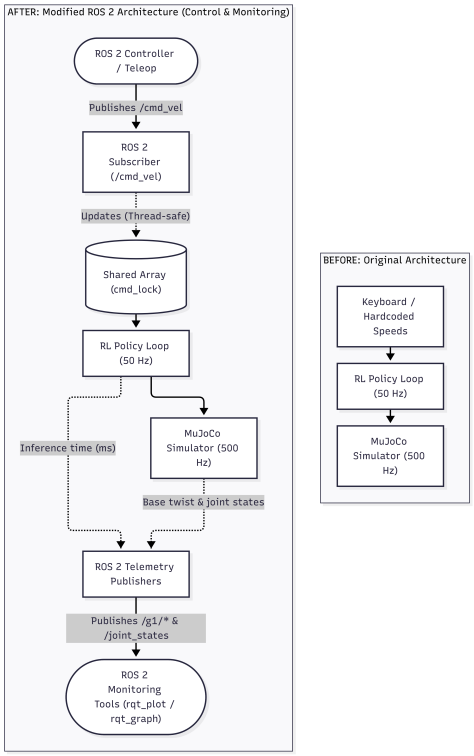


Figure 9: Standalone architecture (before) versus ROS 2-integrated architecture (after).

7.2 Overview of the Integrated Pipeline

The integrated architecture is hosted in `deploy_mujoco_ros.py`, which instantiates `G1BridgeNode`. The node adds a ROS 2 interface to the original loop without changing the policy or the control equations. Figure 10 illustrates the data flow and execution pipeline.

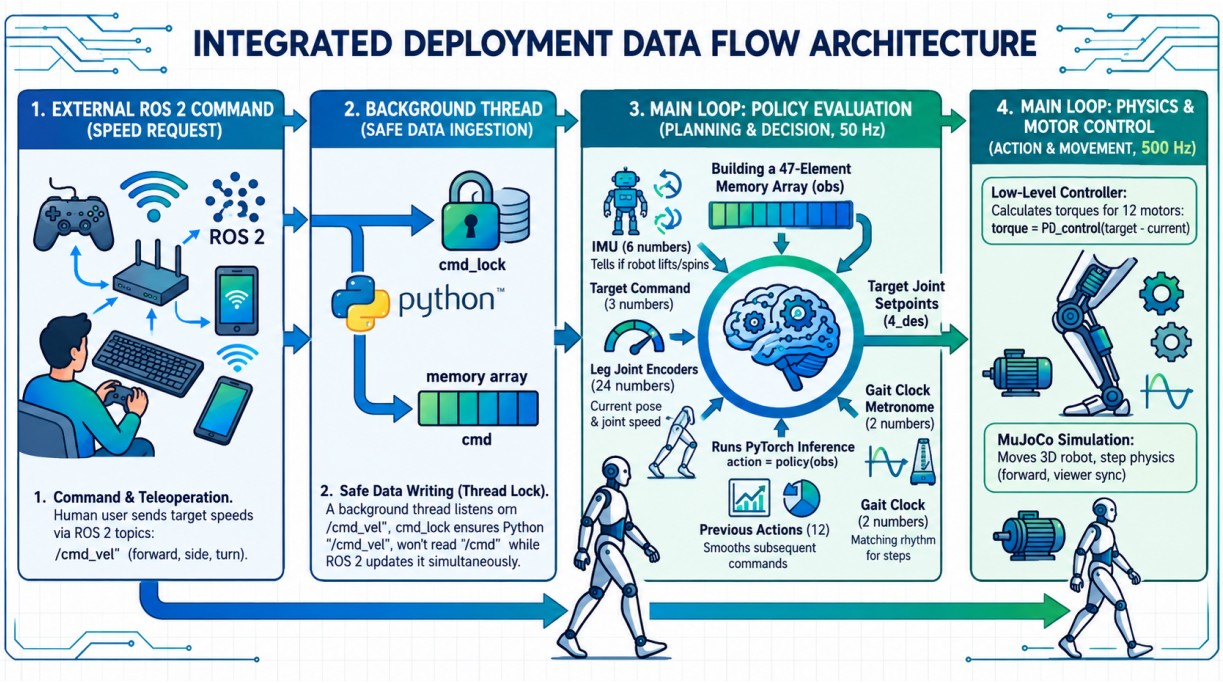


Figure 10: High-level data flow and execution model. External asynchronous ROS 2 commands feed the policy evaluation (50 Hz), which drives the physics and motor control loop (500 Hz).

7.3 Data Flow and Execution Model

The node runs in a **single thread** of a single Python process. At every iteration of the main loop (one physics step of 2 ms), it calls `rclpy.spin_once(timeout_sec=0.0)`, which processes the pending ROS 2 callbacks and returns immediately if there are none. The `/cmd_vel` callback writes the command into the shared array `cmd` = $[v_x, v_y, \omega_z]$ under a mutex (`cmd_lock`), and the policy step reads it under the same mutex. With a single thread the lock is not strictly necessary; it is kept so that the code remains correct if the node is later moved to a multi-threaded executor. The key design point is that **command reception, policy evaluation, and simulation share one execution loop**.

Because everything shares one thread, a slow callback or a slow inference delays the next physics step. The simulated time is not affected, but the simulation may run slower than real time. The real rate can be checked with `ros2 topic hz`. The pipeline is organized into four stages (Figure 11), from **velocity-command reception** through **policy inference and telemetry** to **joint control and physics**.

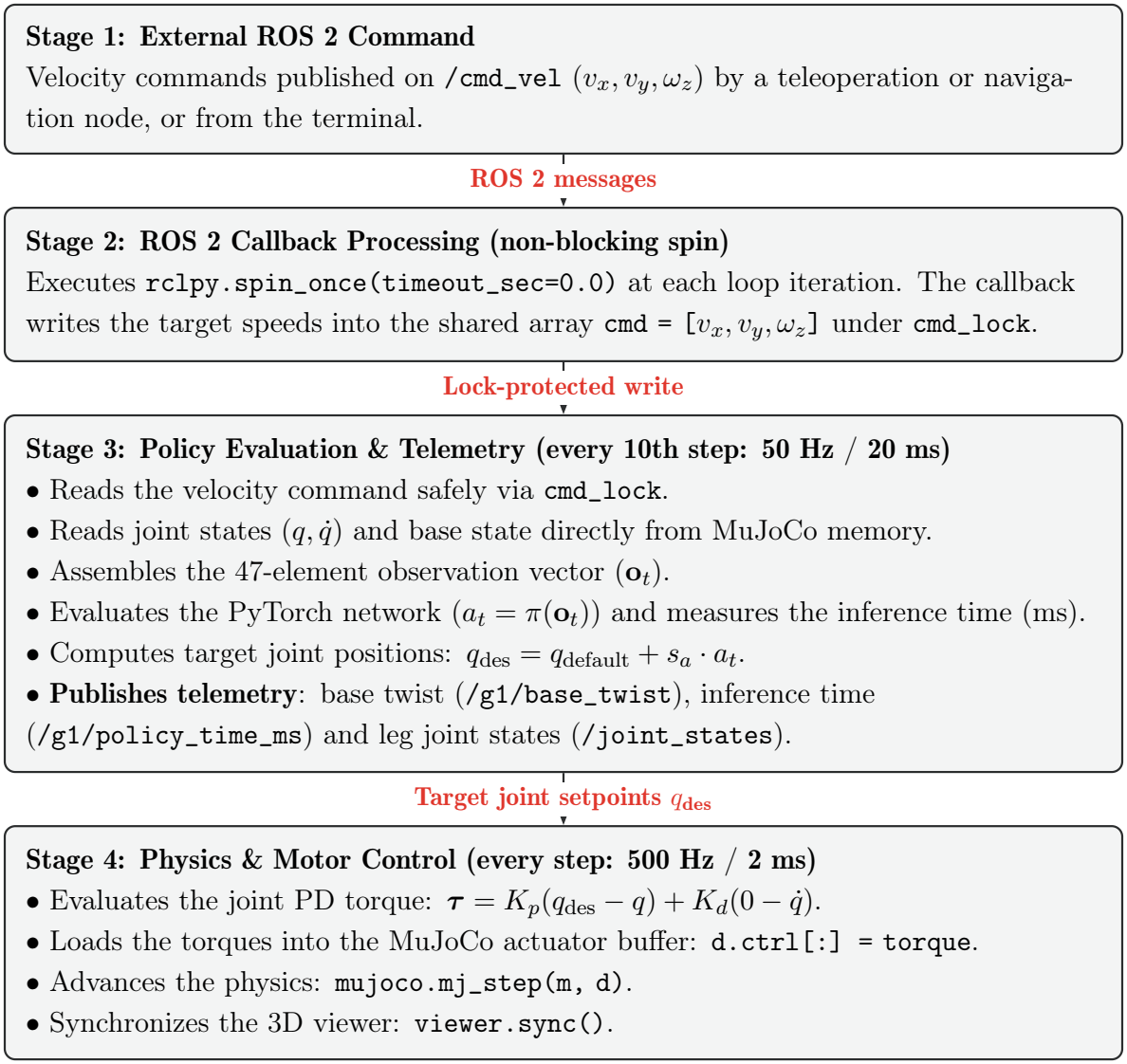


Figure 11: Four-stage data flow architecture and telemetry integration pipeline.

7.4 Breakdown of the Execution Stages

1. Command ingestion and non-blocking spinning (Stages 1 and 2)

- **Subscriber callback:** `G1BridgeNode` subscribes to `/cmd_vel` to receive the target linear (v_x, v_y) and angular (ω_z) velocities.
- **Non-blocking spin:** the main loop calls `rclpy.spin_once(timeout_sec=0.0)` at each iteration, so callbacks are executed without waiting for new messages.
- **Lock guard (`cmd_lock`):** a mutual exclusion lock protects the shared command buffer `cmd`.

The last received command stays in memory until a new message arrives: there is no timeout. If the publisher stops, the robot keeps following the last command. Figure 12 shows the complete communication flow.

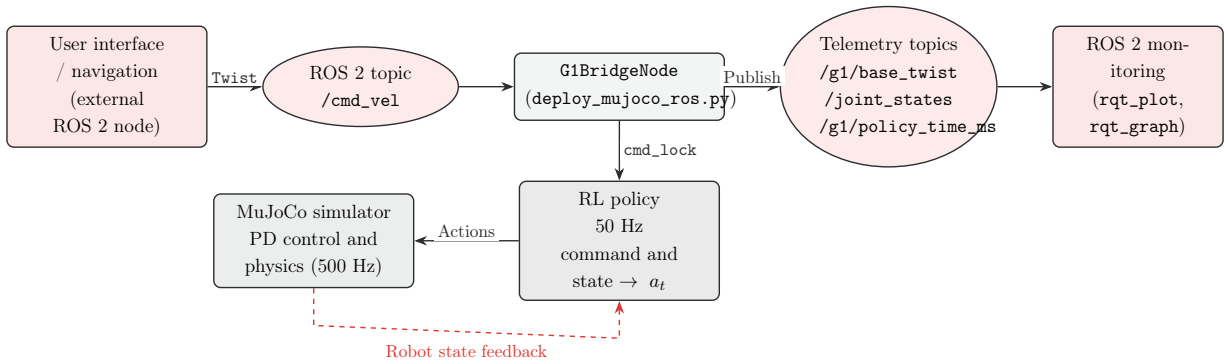


Figure 12: Bi-directional communication flow between ROS 2 command sources, the RL policy, MuJoCo physics, and monitoring tools.

2. Observation vector assembly and policy inference (Stage 3)

Every 20 ms (50 Hz), the node reads the simulator state (`d.qpos`, `d.qvel`) and assembles the 47-element observation $\mathbf{o}_t$ described in Table 2. The network then outputs the joint offsets a_t, converted to targets by

$$q_{\text{des}} = q_{\text{default}} + s_a \cdot a_t, \qquad s_a = 0.25. \tag{4}$$

3. Outbound telemetry (Stage 3)

At each policy step, `G1BridgeNode` publishes three topics (Table 5), described in Section 7.5.

4. Physics and motor control loop (Stage 4)

At every physics step (2 ms, 500 Hz), the PD controller computes

$$\boldsymbol{\tau} = K_p(q_{\text{des}} - q) + K_d(0 - \dot{q}), \tag{5}$$

the torques are written to `d.ctrl[:]`, `mujoco.mj_step(m, d)` advances the simulation and `viewer.sync()` refreshes the display.

Summary of the integrated architecture

Figure 13 summarizes the three execution contexts.

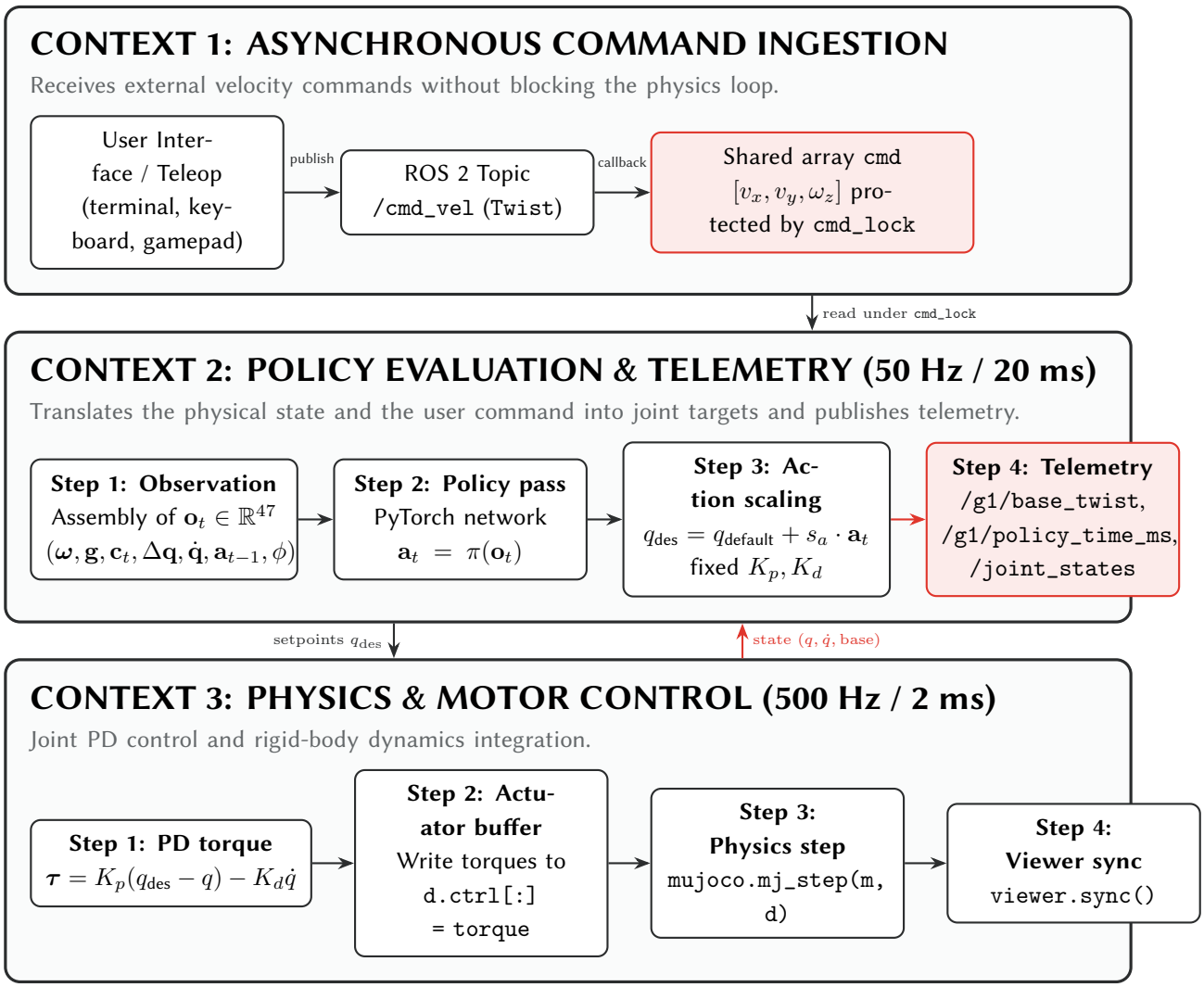


Figure 13: Global summary architecture of data flow and ROS 2 integration for the Unitree G1 in MuJoCo.

7.5 Monitoring

The node publishes its internal state so that the system can be observed with standard ROS 2 tools while it runs. Table 5 lists the topics.

Table 5: ROS 2 topics of `G1BridgeNode`.

Topic	Type	Content
`/cmd_vel` (input)	`geometry_msgs/Twist`	Velocity command (v_x, v_y, ω_z)
`/g1/base_twist`	`geometry_msgs/Twist`	Measured base velocity (linear and angular) from the simulator state
`/g1/policy_time_ms`	`std_msgs/Float32`	Duration of the policy forward pass, in milliseconds
`/joint_states`	`sensor_msgs/JointState`	Position and velocity of the 12 leg joints

The topics are visible from a second terminal:

```
$ ros2 topic list
/cmd_vel
/g1/base_twist
/g1/policy_time_ms
/joint_states
/parameter_events
/rosout
```

While a forward command $v_x = 0.5$ m/s is published, `ros2 topic echo /g1/base_twist` returns a measured linear velocity close to the command (0.493, 0.494 and 0.493 m/s in three consecutive samples). These are instantaneous readings; the commanded and measured velocities are plotted in Figure 17 (Chapter 8), and the joint positions in Figure 14.

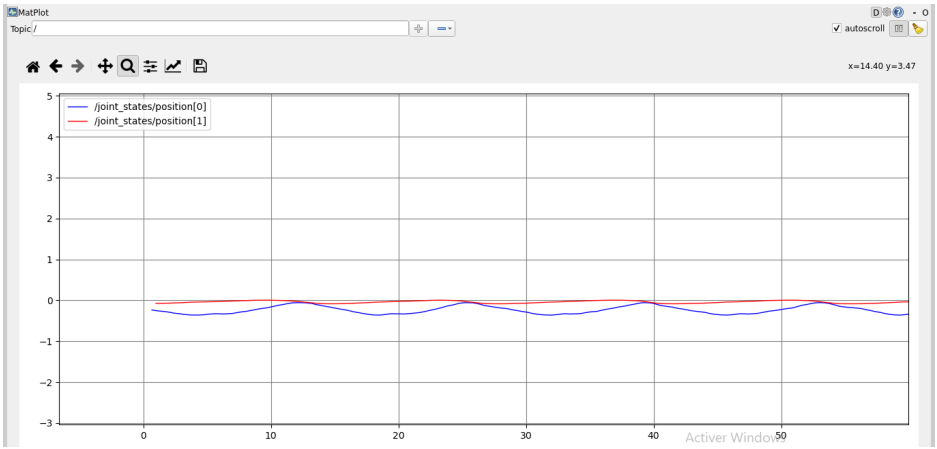


Figure 14: Leg joint positions plotted live with `rqt_plot` during a forward walk.



Figure 15: Node and topic graph of the system, from `rqt_graph`.

7.6 Execution Procedure

Two terminals are used after sourcing the ROS 2 environment: one for the simulation node, one for the commands.

Step 1: Start the simulation node

Terminal 1: simulation node

```
$ python3 deploy_mujoco_ros.py g1.yaml
```

The script initializes the velocity command to zero, so the robot receives no motion request until messages arrive on `/cmd_vel`.

Step 2: Forward command

Terminal 2: forward velocity 0.5 m/s

```
$ ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.5, y:
    0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

The option `-r 10` publishes at 10 Hz. The callback stores $v_x = 0.5$ m/s in the shared buffer, and the policy uses it at its next update.

Step 3: Turning command

Terminal 2: forward and yaw

```
$ ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.3, y:
    0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.5}}"
```

This requests $v_x = 0.3$ m/s and $\omega_z = 0.5$ rad/s.

Step 4: Stop command

Terminal 2: zero velocity

```
$ ros2 topic pub -1 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0, y:
   0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

This sets the requested velocity to zero. Simply stopping the publisher does not stop the robot: the script has no command timeout, so the last received command stays in the buffer. A timeout and a safe-stop behavior are proposed as perspectives.

CHAPTER 8

Tests and Results

8.1 Test Protocol

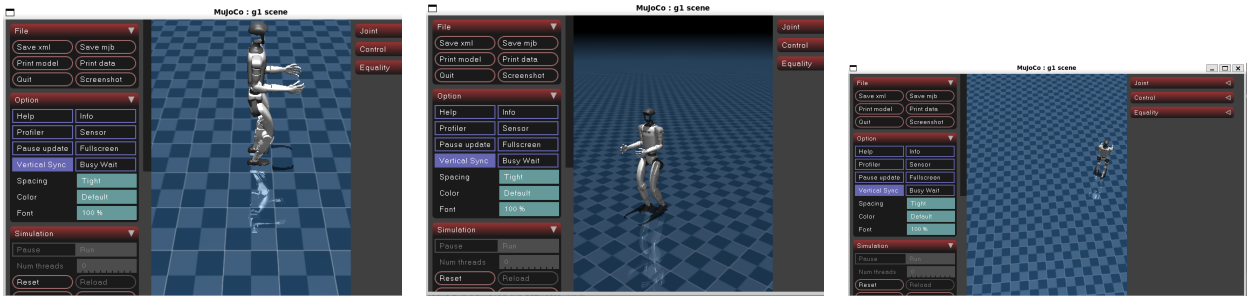
All tests are run in simulation with the pre-trained policy `motion.pt` and the parameters of Tables 3 and 4. The environment is Ubuntu 22.04 on WSL2 with ROS 2 Humble.

On this machine the simulation ran much slower than real time (about 5 %), probably because of the rendering of the viewer under WSL. The times in this chapter are therefore simulation times, except the inference time, which is measured on the computer. The commands and the robot velocity were recorded with `ros2 bag` and analyzed with a short Python script. Thus, the results describe **behavior in simulation, not real-time hardware performance**. Table 6 lists the three scenarios.

Table 6: Test scenarios.

Test	**Command** (v_x, v_y, ω_z)
Standing	$(0, 0, 0)$
Forward walk	$(0.5, 0, 0)$
Turning	$(0.3, 0, 0.5)$

8.2 Simulation Snapshots



(a) Standing, $(0, 0, 0)$ (b) Walking, $(0.5, 0, 0)$ (c) Turning, $(0.3, 0, 0.5)$

Figure 16: MuJoCo frames for the three main commands.

8.3 Velocity Tracking

For the forward walk, the robot was commanded to move at 0.5 m/s. Figure 17 shows the commanded velocity and the measured velocity of the robot, plotted live with `rqt_plot`. The measured speed stays close to the command: its mean value is 0.476 m/s, about 5 % below the command. A small difference of this kind is normal for a learned walking policy, which follows the command only approximately. The test lasted only a few seconds of

simulated time and was run once, so this value is indicative,During the entire test duration, the robot maintained dynamic stability and stayed upright without falling or suffering joint torque saturation.

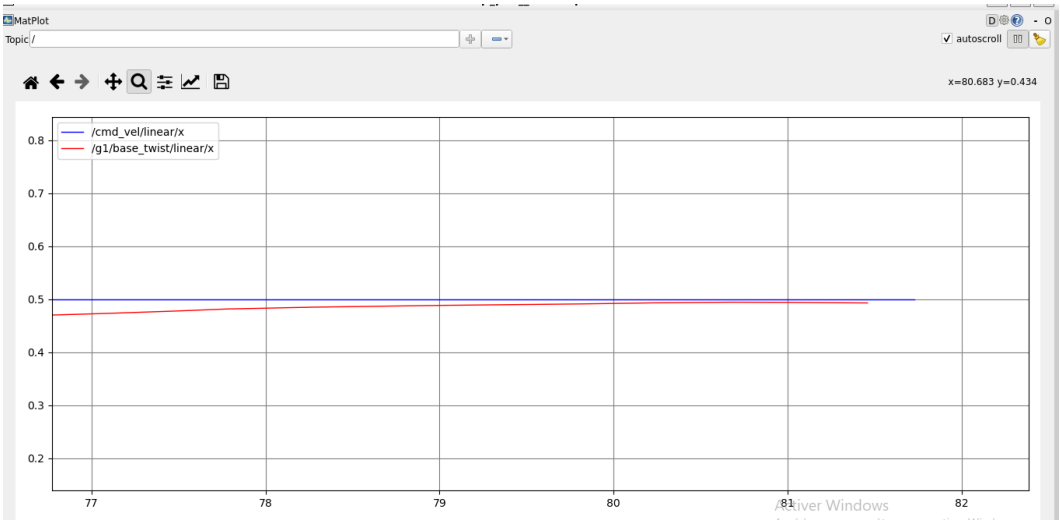


Figure 17: Commanded and measured velocity during the forward walk, plotted with `rqt_plot`.

8.4 Timing Measurements

The time needed by the neural network to compute one action was measured in the script and published on `/g1/policy_time_ms`. On the test machine, the typical time was about 4.4 ms (8.6 ms on average), below the 20 ms available between two policy steps. About 8 % of the steps were slower than 20 ms, with a maximum of 63 ms, because the computer was running the simulation, the viewer and the recording at the same time. These values depend on the test machine; **they were not measured on the robot's onboard computer**.

8.5 Discussion

The tests show that the **ROS 2 layer works as intended**: a **velocity command published on** `/cmd_vel` reaches the policy, the robot walks at a speed close to the command, and its state and inference time can be observed live. Several limitations must be kept in mind:

- **Simulation only.** The policy was not tested on the physical robot.
- **Ideal sensing.** The observation is built from the exact state of the simulator, without sensor noise or delay.
- **Slow simulation.** The real-time factor of about 0.05 prevents any claim about real-time behavior.
- **Limited statistics.** One short run per scenario; no repetition and no variation of terrain.
- **No command timeout.** If the publisher stops, the last command stays active and the robot keeps walking; a timeout would be necessary for safe operation.

- **Single-threaded loop.** A slow callback or inference delays the next physics step.
- **Policy not trained or modified here.** The performance is that of the policy provided by Unitree.

CHAPTER 9

Conclusion and Perspectives

9.1 Conclusion

This internship addressed the integration of a pre-trained reinforcement-learning walking policy for the Unitree G1 into a ROS 2 environment. Starting from the standalone MuJoCo deployment provided with `unitree_rl_gym`, the control pipeline was analyzed: a 47-element observation, a policy evaluated at 50 Hz, and a joint PD controller running at 500 Hz in simulated time. The script was then extended into a ROS 2 node that receives velocity commands on `/cmd_vel` and publishes the measured base velocity, the joint states and the policy inference time for live monitoring with standard tools.

In a forward-walking test at 0.5 m/s in simulation, the measured mean speed was 0.476 m/s; the simulation ran at about 5 % of real time on the test machine.. The results are limited by a short test window, a single run per scenario, a simulation running at about 5 % of real time, and an inference-time distribution with a heavy tail (7.8 % of the steps above 20 ms). The work provides the interface required to connect a navigation stack to the humanoid, which is a natural continuation in the context of the navigation research of the host team. This internship allowed me to bridge theoretical knowledge in automatic control with practical software architecture for complex robotic systems. I gained hands-on experience in ROS 2 node development, thread-safe inter-process communication, physics-engine synchronization in MuJoCo, and practical neural network deployment pipelines for high-DoF humanoid platforms.

9.2 Perspectives

- Deploy the policy on the physical G1 through `unitree_sdk2` / `unitree_ros2`, with safety limits.
- Add a command timeout and a safe-stop behavior.
- Find and remove the cause of the low real-time factor (for example the frequency of the viewer synchronization), then repeat the tests with longer runs and several repetitions.
- Measure the inference time on the embedded computer of the robot.
- Add joint names to `/joint_states` and publish the base pose, to visualize the robot in RViz.
- Connect a navigation stack (Layer 2) to produce `/cmd_vel` automatically.
- Evaluate robustness (pushes, uneven terrain, sensor noise) and compare with a model-based controller.

- Train or fine-tune a policy with `unitree_rl_gym`.

Bibliography

References

[1] CDTA, official website: history, organization, research areas and technology transfer.https://www.cdta.dz/fr/

[2] DGRSDT, *CDTA profile and research framework*; Presidential Decree 88-61 (1988); Executive Decree 03-457 (2003); Law 15-21 (2015).

[3] Unitree Robotics, *G1 Humanoid Robot Overview and Technical Specifications*, 2024.

[4] Quadruped.de, *Unitree G1 EDU Development Computing and Joint Interface Guides*, 2024.

[5] Open Robotics, *ROS 2 Humble Hawksbill Documentation*, 2022.

[6] E. Todorov, T. Erez and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *IEEE/RSJ IROS*, 2012.

[7] J. Schulman et al., "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[8] N. Rudin, D. Hoeller, P. Reist and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Conference on Robot Learning (CoRL)*, 2021.

[9] S. Kajita et al., "The 3D Linear Inverted Pendulum Mode: A simple modeling for a biped walking pattern generation," in *IEEE/RSJ IROS*, 2001.

[10] C. Cadena et al., "Past, present, and future of simultaneous localization and mapping," *IEEE Transactions on Robotics*, 2016.

[11] J. Zhang and S. Singh, "LOAM: Lidar Odometry and Mapping in Real-time," in *Robotics: Science and Systems*, 2014.

[12] A. Elfes, "Using occupancy grids for mobile robot perception and navigation," *Computer*, 1989.

[13] P. Hart, N. Nilsson and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Systems Science and Cybernetics*, 1968.

[14] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. Journal of Robotics Research*, 2011.

[15] A. Brohan et al., "RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control," arXiv:2307.15818, 2023.

[16] M. Kim et al., "OpenVLA: An Open-Source Vision-Language-Action Model," arXiv:2406.09246, 2024.

[17] Unitree Robotics, `unitree_rl_gym`, GitHub repository, https://github.com/unitreerobotics/unitree_rl_gym.

[18] Unitree Robotics, `unitree_ros2`, GitHub repository, https://github.com/unitreerobotics/unitree_ros2.